

Introduction to PHP

for Dynamic Web Projects

Web Techniques

Prof. Soufiane Hourri

Learning Objectives

By the end of this 2-hour session, students will be able to:

PHP & Core Syntax

- Explain what PHP is and how it runs on a web server
- Set up a local server with XAMPP or WAMP
- Write correct PHP syntax: tags, variables, operators
- Use conditions (if / elseif / switch) and loops
- Build and use indexed and associative arrays

Forms · Database · Sessions

- Create HTML forms and process data with `$_POST` and `$_GET`
- Validate and sanitize user input safely
- Connect PHP to MySQL and understand the 3-step workflow
- Execute INSERT, SELECT, UPDATE, and DELETE queries
- Implement sessions for login, page protection, and logout

1

Part 1

What is PHP?

Server-side scripting · how the web works

What is PHP?

Definition

- PHP stands for : Hypertext Preprocessor.
- It is a recursive acronym, meaning the first letter of the abbreviation refers back to the abbreviation itself.
- Originally, when the language was first created by Rasmus Lerdorf in 1994, the acronym stood for Personal Home Page (or Personal Home Page Tools). As the language evolved into a more powerful, general-purpose scripting tool, the name was changed to reflect its broader utility in web development.

What is PHP?

PHP is Server-Side

PHP code runs on the web SERVER, not in the browser. The user never sees your PHP code — they only receive the final HTML that PHP produces.

PHP is a Scripting Language

You write .php files that execute when a user requests a page. PHP can generate different HTML for every user — dynamic pages.

PHP Connects to Databases

PHP is the bridge between your HTML pages and MySQL. It reads form data, queries the database, and builds the HTML response.

Browser requests page.php → Apache runs PHP code → PHP queries MySQL DB → PHP builds HTML → Browser shows page

HTML / CSS / JS vs PHP

Understanding where each language runs is the key to using them correctly.

	Client-side (Browser)	Server-side (PHP)
Runs on...	The user's browser	The web server (Apache + PHP)
User sees...	The actual source code	Only the HTML output – never PHP
Used for...	Layout, style, interactivity	Logic, databases, logins, forms
File saved...	.html .css .js	.php (inside htdocs/www)
Example	<code><h1>Hello</h1></code>	<code><?php echo 'Hello'; ?></code>

2

Part 2

Getting Started

Install XAMPP · localhost · First PHP file

Running PHP Locally — XAMPP / WAMP

XAMPP installs Apache, PHP, and MySQL in one step. All free, all open-source.

1

Download & Install XAMPP

Go to apachefriends.org
Download XAMPP for your OS
Run the installer (keep all defaults)

2

Start Apache & MySQL

Open XAMPP Control Panel
Click Start next to Apache
Click Start next to MySQL

3

Place files in htdocs

Navigate to:
C:\xampp\htdocs\
Create a project folder:
myapp
Save all .php files inside it

4

Open in browser

Type in the browser bar:
<http://localhost/myapp/index.php>
PHP executes and shows result

Try it now: Create `index.php` in `htdocs` → type `<?php echo 'Hello World!'; ?>` → open <http://localhost/index.php>

3

Part 3

PHP Basics

Syntax · Variables · Conditions · Loops · Arrays

PHP Tags, Echo & Comments

Every PHP file starts with <?php — everything inside is executed by the server.

```
<?php
// Single-line comment (PHP ignores this line)

/*
 * Multi-line comment.
 * Use it for longer explanations.
 */

// echo: outputs text or HTML to the browser
echo "Hello, students!";
echo "<br>";

// Joining strings with a dot .
$teacher = "Prof. Hourri";
echo "Welcome, " . $teacher . "!";

// You can output HTML tags directly
echo "<h2>My First PHP Page</h2>";
echo "<p>This paragraph is generated by PHP.</p>";
?>
```

Line by Line

- 1 <?php opens PHP mode. The server executes everything inside until ?>
- 2 echo sends text or HTML to the browser — it's how PHP produces output.
- 3 Every statement must end with a semicolon ; — this is required!
- 4 // is a single-line comment — PHP ignores it. Use it for notes.
- 5 /* ... */ is a multi-line comment for longer explanations.
- 6 Strings are joined with a dot . — NOT + like in JavaScript.

Variables & Data Types

A variable stores a value in memory. In PHP, every variable starts with a \$ sign.

```
<?php
// Variable = $ + name + value
$name      = "Alice";    // string (text in quotes)
$age       = 21;         // integer (whole number, no quotes)
$average   = 15.50;      // float (decimal number)
$passed    = true;       // boolean (true or false)

// Double quotes: variable IS replaced inside the string
echo "Student: $name";   // → Student: Alice

// Single quotes: variable is NOT replaced
echo 'Student: $name';   // → Student: $name (literal)

// Concatenation: joining with a dot
echo "Age: " . $age . " years old"; // → Age: 21 years old

// Constants: defined once, never change
define("MAX_GRADE", 20);
echo MAX_GRADE;          // → 20
?>
```

What to Remember

- 1 Variables start with \$ — always. No spaces or special chars in the name.
- 2 PHP auto-detects the type: string, integer, float, boolean — no need to declare.
- 3 Double quotes "\$name" replace the variable with its value automatically.
- 4 Single quotes '\$name' do NOT replace — the \$name is shown literally.
- 5 Variable names are case-sensitive: \$Name and \$name are TWO different variables.
- 6 define() creates a constant whose value can never be changed after creation.

Conditions — if / elseif / else / switch

Conditions let PHP make decisions. Essential for login checks, role-based pages, and validation.

```
<?php
$grade = 14;

// if / elseif / else
if ($grade >= 16) {
    echo "Very Good";
} elseif ($grade >= 12) {
    echo "Satisfactory";    // ← this one runs
} elseif ($grade >= 10) {
    echo "Average";
} else {
    echo "Failed";
}

// switch: cleaner when checking one variable many times
$role = "admin";
switch ($role) {
    case "admin":
        echo "Admin Dashboard";
        break;    // stops checking other cases
    case "student":
        echo "Student Profile";
        break;
    default:
        echo "Unknown role";
}
```

How Conditions Work

- 1 if (\$condition) — runs the block ONLY when the condition is true.
- 2 PHP checks elseif blocks in ORDER. It stops at the first true one.
- 3 else is the fallback — runs when no condition above matched.
- 4 Operators: == != >= <= > < and === (checks type too).
- 5 switch is cleaner than many elseif blocks when checking ONE variable.
- 6 break is required in switch — without it, all cases below would also run.

Loops — for / while / foreach

Loops repeat a block of code. foreach is the most used in PHP — perfect for displaying database rows.

```
<?php
// for: best when you know the count in advance
for ($i = 1; $i <= 5; $i++) {
    echo "Row: $i <br>";
} // → Row: 1, Row: 2 ... Row: 5

// while: runs as long as the condition is true
$n = 0;
while ($n < 3) {
    echo "Attempt: $n <br>";
    $n++; // ⚠ always increment to avoid infinite loop!
}

// foreach: loops through an array — most used!
$modules = ["PHP", "MySQL", "HTML", "CSS"];
foreach ($modules as $module) {
    echo $module . "<br>";
}
// → PHP, MySQL, HTML, CSS (one per line)

// foreach with key => value (associative array)
$student = ["name" => "Alice", "grade" => 18];
foreach ($student as $key => $value) {
    echo "$key: $value <br>";
} // → name: Alice   grade: 18
?>
```

Choosing the Right Loop

- 1** for: 3 parts — init (\$i=1), condition (\$i<=5), increment (\$i++). Runs a fixed number of times.
- 2** while: continues as long as condition is true. Always change the variable inside or it loops forever!
- 3** foreach is the MOST useful — perfect for displaying database results.
- 4** foreach (\$arr as \$item) — \$item holds one value per iteration.
- 5** foreach (\$arr as \$key => \$value) gives you both key and value.
- 6** After the loop ends, PHP continues normally with the next line of code.

Arrays — Storing Multiple Values

An array holds a list of values in one variable. DB query results come back as arrays.

```
<?php
// — INDEXED ARRAY (numeric keys) —————
$subjects = ["PHP", "MySQL", "HTML", "CSS"];
echo $subjects[0];      // → PHP   (first item, index 0)
echo $subjects[2];      // → HTML
echo count($subjects);  // → 4
$subjects[] = "JS";     // Append item at the end

// — ASSOCIATIVE ARRAY (string keys) —————
$student = [
    "name" => "Alice",      // key => value
    "email" => "alice@etu.ma",
    "grade" => 18,
];
echo $student["name"];    // → Alice
echo $student["grade"];  // → 18

// — ARRAY OF ARRAYS (like DB table rows) —
$class = [
    ["name" => "Alice", "grade" => 18],
    ["name" => "Bob",   "grade" => 15],
];
foreach ($class as $row) {
    echo $row["name"] . ": " . $row["grade"] . "<br>";
}
// → Alice: 18      Bob: 15
?>
```

Why Arrays Matter

- 1 Indexed: items accessed by number starting from 0. `$arr[0]` is the first item.
- 2 `count($array)` returns the total number of items in the array.
- 3 `$array[] = value` appends a new item to the end without a key.
- 4 Associative: uses text keys — just like column names in a database.
- 5 When SELECT returns rows from MySQL, each row IS an associative array.
- 6 An array of arrays = a table — loop it with `foreach` to display all rows.

4

Part 4

Forms & User Input

`$_GET` · `$_POST` · Validation

GET vs POST — Two Ways to Send Data

An HTML form sends data to PHP using GET or POST. Choosing the right one matters for security.

method="GET"

- Data is visible in the URL:
- `search.php?query=php&page=2`
- Can be bookmarked or shared as a link
- Use for: search, filters, pagination
- NEVER use for passwords or private data
- Read in PHP with: `$_GET["query"]`

method="POST"

- Data is hidden inside the HTTP request body
- Cannot be bookmarked (intentional)
- Use for: login, register, add/edit/delete
- Safer for sensitive data like passwords
- Read in PHP with: `$_POST["email"]`

Processing Form Data in PHP

The form collects data. The PHP file specified in `action=""` reads and uses it.

```
<!-- — register.html: the HTML form — -->
<form action="process.php" method="POST">
  <input type="text"      name="name"  placeholder="Full Name">
  <input type="email"    name="email"  placeholder="Email">
  <input type="password" name="pass"   placeholder="Password">
  <button type="submit">Register</button>
</form>

<?php
/* — process.php: reads the submitted data — */

// Step 1: Make sure the form was submitted via POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Step 2: Read each field by its "name" attribute
    $name  = $_POST["name"];
    $email = $_POST["email"];
    $pass  = $_POST["pass"];

    // Step 3: Use the values
    echo "Hello, " . $name;
    echo "Email: " . $email;
}
?>
```

Step by Step

- 1 `action="process.php"` — form data is sent to this PHP file on submit.
- 2 `method="POST"` — data travels hidden in the request body, not the URL.
- 3 `name="name"` — the HTML name attribute is how PHP identifies each field.
- 4 `$_SERVER["REQUEST_METHOD"]` — tells you HOW the page was opened (GET or POST).
- 5 `$_POST["name"]` — reads the value the user typed in the field named `name`.
- 6 Always wrap POST code in the `REQUEST_METHOD` check — avoids errors on first load.

Input Validation — Never Trust User Data

Always check and clean data before using it. Users can submit empty fields, wrong formats, or harmful input.

```
<?php
$error = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Step 1: Clean inputs – remove leading/trailing spaces
    $name = trim($_POST["name"]);
    $email = trim($_POST["email"]);
    $pass = $_POST["pass"];

    // Step 2: Check required fields
    if (empty($name)) {
        $error = "Name is required.";
    }

    // Step 3: Validate format
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = "Invalid email format.";
    }

    // Step 4: Check password length
    } elseif (strlen($pass) < 6) {
        $error = "Password must be 6+ characters.";
    }

    // Step 5: Only save if no errors
    if (empty($error)) {
        echo "Input is valid – saving to database...";
    } else {
        echo "Error: " . $error;
    }
}
?>
```

Essential Functions

- 1 trim() removes spaces from start and end of a string. Always do this first.
- 2 empty() returns true if the value is "" 0 null or false.
- 3 filter_var(, FILTER_VALIDATE_EMAIL) checks email format — built in, reliable.
- 4 strlen() counts characters — use it to enforce minimum lengths.
- 5 Collect the FIRST error and stop — show it to the user to fix before saving.
- 6 htmlspecialchars() converts < > & to safe text — protects against XSS attacks.

5

Part 5

MySQL & Databases

Connect · Query · Fetch · INSERT · SELECT · UPDATE · DELETE

Why Do We Need a Database?

PHP variables only live during one page load. A database stores data permanently between requests.

Problem Without a DB

A user fills a form — PHP reads the data — but when the page reloads, everything is GONE. No memory between page loads.

Solution: MySQL

MySQL saves data permanently on disk. PHP writes data IN with INSERT and reads data OUT with SELECT — across any page, any user, any time.

Your Project Tables

Your DB will have tables like: users, students, products, orders

Each table is a spreadsheet. Rows are records, columns are fields.

Setup before writing PHP:

Open phpMyAdmin at <http://localhost/phpmyadmin> → Create database school → Create table students (id AUTO_INCREMENT, name, email, phone) → Set id as PRIMARY KEY

The mysqli Workflow — 3 Essential Steps

Every PHP database interaction follows the same 3 steps: Connect → Query → Fetch.

Step 1 CONNECT

`mysqli_connect()`

Open a connection to the MySQL server.

Provide: host, user, password, database name.

Always test that the connection succeeded.

Step 2 QUERY

`mysqli_query()`

→ Send a SQL command to the database. Used for: SELECT, INSERT, UPDATE, DELETE. Returns a result object (for SELECT) or true/false.

Step 3 FETCH

`mysqli_fetch_*`

→ Read rows from the result object one by one. Only needed after SELECT — not INSERT/UPDATE. Loop with `while` to fetch all rows.

```
$conn = mysqli_connect(...);    ←step 1    $result = mysqli_query($conn, "SELECT...");    ←step 2    $row = mysqli_fetch_assoc($result);    ←step 3
```

Step 1 — Connecting to MySQL

Save this file as `db_connect.php` and include it on every page that needs the database.

```
<?php
/* — db_connect.php ————— */

// Define the connection parameters
$host      = "localhost"; // always localhost on XAMPP
$user      = "root";      // default XAMPP username
$password  = "";          // empty by default in XAMPP
$database  = "school";    // the DB you created in phpMyAdmin

// Open the connection with mysqli_connect()
$conn = mysqli_connect($host, $user, $password, $database);

// ALWAYS check that the connection succeeded!
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Connection is open — ready to send queries.
// On every page that needs the DB, add:
//     require_once("db_connect.php");
?>
```

Understanding Each Part

- 1 `mysqli_connect()` opens a connection to MySQL. It takes 4 arguments.
- 2 "localhost" means the DB server is on the same machine (true on XAMPP).
- 3 "root" is XAMPP's default MySQL user. Password is empty by default.
- 4 `!$conn` means "if connection FAILED" — the `!` reverses the boolean.
- 5 `die()` stops the script immediately and shows the error. Critical for debugging.
- 6 `require_once()` includes the file exactly once — safe to call many times.

Step 3 — The 3 Ways to Fetch Data

After a *SELECT* query, PHP gives you a result object. You must *FETCH* rows from it. There are 3 ways:

```
$result = mysqli_query($conn, "SELECT id, name, grade FROM students"); // result has rows – now fetch them:
```

`mysqli_fetch_assoc()`

Recommended ✓

```
$row = mysqli_fetch_assoc($result);
// Returns associative array (key =
// column name)
// [
//   "id"    => 1,
//   "name"  => "Alice",
//   "grade" => 18
// ]
echo $row["name"]; // → Alice
echo $row["grade"]; // → 18
```

Uses column names as keys — readable and safe. This is the recommended choice for beginners and professionals alike.

`mysqli_fetch_row()`

Numeric index only

```
$row = mysqli_fetch_row($result);
// Returns numeric indexed array only
// [0 => 1, 1 => "Alice", 2 => 18]

echo $row[0]; // → 1      (id)
echo $row[1]; // → Alice  (name)
echo $row[2]; // → 18     (grade)
```

Accesses by column position (0, 1, 2...).
Fragile: if you change column order in your *SELECT*, all indexes break.

`mysqli_fetch_array()`

Both (use with care)

```
$row = mysqli_fetch_array($result);
// Returns BOTH associative AND
// numeric
// [0=>"Alice", "name"=>"Alice", ...]

echo $row["name"]; // → Alice
echo $row[1];      // → Alice (same!)
```

Returns double the data (both keys). Uses more memory. Only useful in rare cases — avoid it unless you have a specific need.

Step 2 — INSERT: Adding Data to the Database

After validating form data, use INSERT INTO to save a new row in your table.

```
<?php
require_once("db_connect.php"); // Step 1: connect

// Read and clean the form values
$name = trim($_POST["name"]);
$email = trim($_POST["email"]);

// Step 2: build the SQL INSERT statement
// Column names must match your table exactly!
$sql = "INSERT INTO students (name, email)
        VALUES ('$name', '$email')";

// Step 2: execute the query
if (mysqli_query($conn, $sql)) {

    // ✓ Success: redirect to the list page
    header("Location: list.php");
    exit; // always exit after header()

} else {
    // ✗ Something went wrong — show DB error
    echo "Error: " . mysqli_error($conn);
}

// mysqli_insert_id() gives the ID of the new row
$new_id = mysqli_insert_id($conn);
echo "New student ID: " . $new_id;
?>
```

How INSERT Works

- 1 INSERT INTO students — specifies which table receives the new row.
- 2 (name, email) — the columns you're filling, in parentheses. Match your table!
- 3 VALUES ('\$name', '\$email') — the actual data in the same order as the columns.
- 4 mysqli_query(\$conn, \$sql) runs the query. Returns true on success, false on fail.
- 5 header("Location: list.php") redirects the user. Always call exit right after.
- 6 mysqli_insert_id(\$conn) returns the AUTO_INCREMENT id of the row just inserted.

Step 2+3 — SELECT: Reading & Displaying Data

SELECT retrieves rows. Then you loop with while + fetch_assoc to display each row as HTML.

```
<?php
require_once("db_connect.php");

// Step 2: Run the SELECT query
$result = mysqli_query($conn,
    "SELECT * FROM students ORDER BY id DESC"
);

// How many rows did we get?
$total = mysqli_num_rows($result);
echo "Total students: " . $total . "<br>";

// Build an HTML table
echo "<table border='1'>";
echo "<tr><th>ID</th><th>Name</th><th>Email</th></tr>";

// Step 3: Fetch one row at a time until no rows left
while ($row = mysqli_fetch_assoc($result)) {
    echo "<tr>";
    echo "<td>" . $row["id"] . "</td>";
    echo "<td>" . $row["name"] . "</td>";
    echo "<td>" . $row["email"] . "</td>";
    echo "<td><a href='delete.php?id=" . $row["id"] .
    "'>Delete</a></td>";
    echo "</tr>";
}
echo "</table>";
?>
```

How SELECT + Fetch Works

- 1 SELECT * means all columns. Replace * with names (id, name) for better practice.
- 2 ORDER BY id DESC — newest rows first. Use ASC for oldest first.
- 3 mysqli_num_rows(\$result) — number of rows returned. Check before looping.
- 4 while (\$row = mysqli_fetch_assoc(\$result)) — fetches one row per iteration.
- 5 The loop continues until fetch returns NULL (no more rows) — works for any count.
- 6 \$row["id"] — access column values by name. Use the exact column name from the DB.

UPDATE & DELETE — Modifying and Removing Data

UPDATE changes existing rows. DELETE removes them. Always use WHERE or you affect ALL rows!

UPDATE

```
<?php
require_once("db_connect.php");

// Read new values from the form
$id          = $_GET["id"];           // ID from URL
$new_name    = trim($_POST["name"]);
$new_grade   = trim($_POST["grade"]);

// UPDATE: change specific columns for one row
$sql = "UPDATE students
        SET name='$new_name',
            grade='$new_grade'
        WHERE id='$id'";

if (mysqli_query($conn, $sql)) {
    echo "Student updated!";
    // How many rows were changed?
    $changed = mysqli_affected_rows($conn);
    echo "Rows affected: " . $changed;
} else {
    echo "Error: " . mysqli_error($conn);
}
```

DELETE

```
<?php
require_once("db_connect.php");

// Get the ID from the URL: delete.php?id=3
$id = $_GET["id"];

// DELETE: remove the row with this id
$sql = "DELETE FROM students
        WHERE id='$id'";

if (mysqli_query($conn, $sql)) {
    echo "Student deleted!";
    header("Location: list.php");
    exit;
} else {
    echo "Error: " . mysqli_error($conn);
}

// ⚠ DANGER — deletes ALL rows (no WHERE):
// "DELETE FROM students" ← NEVER do this!
?>
```

⚠ Always use WHERE id='\$id' — without it, UPDATE or DELETE will affect EVERY row in the table!

6

Part 6

Sessions & Login

`session_start()` · `$_SESSION` · Protect Pages

Sessions — Keeping Users Logged In

A session lets PHP remember who the user is across multiple pages. Without it, every page is anonymous.

```
<?php
/* — login.php: verify and start session — */
session_start(); // ← MUST be line 1, before any output!

require_once("db_connect.php");
$email = $_POST["email"];
$pass  = $_POST["password"];

$sql = "SELECT * FROM users WHERE email='$email' LIMIT 1";
$user = mysqli_fetch_assoc(mysqli_query($conn, $sql));

if ($user && $user["password"] == $pass) {
    // ✓ Login OK — store info in the session
    $_SESSION["user_id"] = $user["id"];
    $_SESSION["username"] = $user["name"];
    $_SESSION["logged_in"] = true;
    header("Location: dashboard.php"); exit;
} else {
    echo "Wrong email or password.";
}

/* — dashboard.php: protect the page — */
session_start();
if (!isset($_SESSION["logged_in"]) || !$_SESSION["logged_in"]) {
    header("Location: login.php"); exit;
}
echo "Welcome, " . $_SESSION["username"] . "!";

/* — logout.php — */
session_start();
session_destroy(); // clears all session data
header("Location: login.php"); exit;
?>
```

How Sessions Work

- 1 session_start() MUST be the very first line — before any echo or HTML output.
- 2 PHP stores session data on the SERVER. The browser only keeps a session ID cookie.
- 3 \$_SESSION["key"] = value — saves data that persists across all pages of the site.
- 4 On protected pages: check \$_SESSION["logged_in"] and redirect if not set.
- 5 Always call exit after header("Location:...") — PHP keeps running otherwise!
- 6 session_destroy() deletes ALL session data — the user is completely logged out.

How This Maps to Your Project

Every concept you learned today maps directly to pages you need to build.

Login Page

POST form + SELECT + verify password + session_start() + \$_SESSION

Register Page

POST form + validate all fields + INSERT INTO users + redirect

Dashboard

session_start() + check \$_SESSION["logged_in"] + SELECT to load data

Add Record

POST form + validate + INSERT INTO your_table + redirect

View All Records

SELECT * + mysqli_fetch_assoc() loop + HTML table with rows

Edit / Delete

GET id from URL + UPDATE or DELETE WHERE id=\$id + redirect

Mini Classroom Activity · 15 Minutes

Build a Student Registration Mini-App

1

Create the HTML form

Fields: Full Name, Email, Phone | method="POST" | action="save.php"

2

Set up the database

In phpMyAdmin: DB = school | Table: students (id AUTO_INCREMENT, name, email, phone)

3

Validate inputs

In save.php: check name not empty, email valid format, phone not empty

4

INSERT into database

Include db_connect.php, read \$_POST values, run INSERT INTO students

5

SELECT and display

Below the form: SELECT * FROM students + while + mysqli_fetch_assoc() + HTML table

Common Mistakes to Avoid

X

Missing semicolons

Every PHP statement must end with ;

X

Wrong DB name or credentials

Check \$host, \$user, \$password, \$database carefully

X

Confusing GET and POST

Login/register → POST | search/filter → GET

X

No WHERE in UPDATE/DELETE

Without WHERE, ALL rows are modified or deleted!

X

Forgetting session_start()

Must be line 1 — before any echo or HTML output

X

Not checking with isset()

Always use `isset()` before reading `$_POST` or `$_GET`

X

echo before header()

`header()` must come before any echo or HTML output

X

Not validating before INSERT

Always `trim()` and check `empty()` before saving to DB

Project Tips

01

Start simple — one working page is worth more than five broken ones. Build page by page.

02

Design your database schema BEFORE writing any PHP. Know your tables and column names first.

03

Create `db_connect.php` once and `require_once()` it on every page that needs the database.

04

Test each feature before adding the next. Confirm the form works before adding validation.

05

Get a working CRUD (Create, Read, Update, Delete) first — then add design and styling.

06

Name files clearly: `login.php`, `register.php`, `add_student.php` — not `test1.php`, `page2.php`.

07

Keep PHP logic at the top of the file and HTML output below. Much easier to read and fix.

08

If something breaks: check semicolons, DB column names, and `isset()` — 90% of bugs are there.

Session Recap & Homework

What We Covered Today

- What PHP is — server-side scripting
- XAMPP setup: localhost & htdocs
- Tags, echo, comments, variables, types
- Conditions: if / elseif / switch
- Loops: for / while / foreach
- Arrays: indexed and associative
- Forms: POST vs GET + \$_POST
- Validation: empty, trim, filter_var
- mysqli: 3-step workflow (connect→query→fetch)
- 3 fetch functions: assoc / row / array
- INSERT, SELECT, UPDATE, DELETE
- Sessions: login, protect pages, logout

Homework

Build a complete mini PHP application:

1. Create DB school — table students (id, name, email, phone)
 2. Build register.php: POST form + validate inputs
 3. INSERT valid data into the students table
 4. Build list.php: SELECT all + while loop + HTML table
 5. Build edit.php: UPDATE a student's data by id
 6. Build delete.php: DELETE a student by id
- ★ BONUS: login.php + session + protect all pages